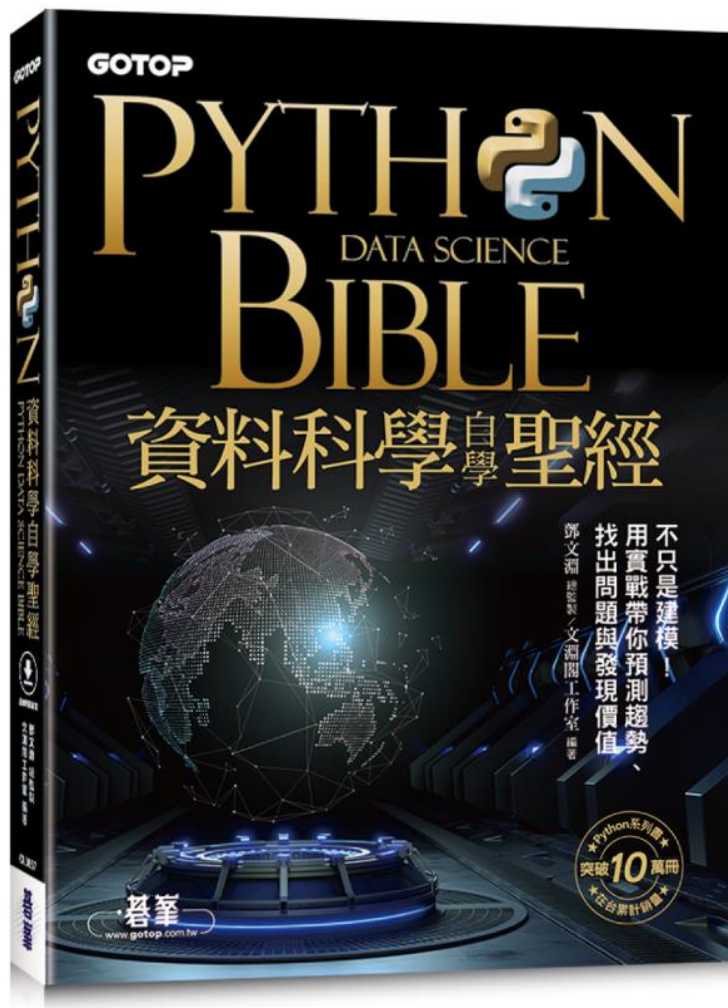




PYTHON DATA SCIENCE BIBLE 資料科學自學聖經

碁峯資訊

【投影片使用規範與聲明】本投影片僅供非營利教學用途，教師得搭配用書授課講解使用，可於用書期間內將投影片置放於學校內部網站，但需有帳密權限機制，且僅供修課學生瀏覽使用。敬請老師善盡著作權保護之責，請勿將投影片任意散布與販售，亦不得以任何形式或方法轉載內容使用。

資料科學神器： Numpy 與Pandas

[2.1 Numpy：高速運算的解決方案](#)

[2.2 Numpy 陣列建立](#)

[2.3 Numpy 陣列取值](#)

[2.4 Numpy 的陣列運算功能](#)

[2.5 Pandas：資料處理分析的強大工具](#)

[2.6 Series 的使用](#)

[2.7 DataFrame 的建立](#)

[2.8 Pandas DataFrame 資料取值](#)

[2.9 DataFrame 資料操作](#)

2.1 Numpy：高速運算的解決方案

列與矩陣的運算，也提供了相關運算的數學函式庫。

2.1.1 安裝Numpy 與載入模組

可以使用下列指令在Python 中安裝Numpy：

```
!pip install numpy
```

使用前請先載入Numpy 模組，一般為了能在使用時更加方便，會設定別名np：

```
import numpy as np
```

2.1.2 認識Numpy 陣列

Numpy 使用ndarray (N-dimensional array) 陣列來取代Python 的串列資料，這是一個可以裝載**相同類型資料**的多維容器，其中的維度、大小及資料類型分別由ndim、shape 及dtype 屬性來定義。

在Numpy 陣列中，一維陣列為 **向量(vector)**，二維陣列為 **矩陣(matrix)**，而陣列是以各 **軸向(axis)** 的數量來代表陣列的 **形狀(shape)**。以下圖為例分別說明：



2.2 Numpy 陣列建立

2.2.1 建立基本陣列

建立陣列：np.array()

可以使用np.array() 函式，利用串列(List) 或是元組(Tuple) 來建立陣列，語法如下：

```
np.array( 串列或元組 , dtype= 資料格式 )
```

注意：dtype 參數可以設定資料的格式，預設值為int64。

建立有序整數陣列：np.arange()

np.arange() 函式與range() 函式的方法相似，可以建立等距的整數陣列，語法如下：

```
np.arange([ 起始值 , ] 終止值 [, 間隔值 ])
```

注意：使用 np.arange() 函式設定範圍，預設起始值是 0，間隔值是 1，終止值是指到終止值前，不包含終止值喔！

建立等距陣列：np.linspace()

np.linspace() 函式可以設定一個範圍的等距陣列，語法如下：

```
np.linspace( 起始值 , 終止值 , 元素個數 )
```

注意：np.linspace() 函式返回值陣列的元素的資料型別是 float，設定的範圍有包含起始值及終止值。

建立同值為0 的陣列：np.zeros()

np.zeros() 函式可以根據設定的形狀建立全部都是0 的陣列，語法為：

```
np.zeros( 陣列形狀 )
```

建立同值為1 的陣列：np.ones()

np.ones() 函式可以根據設定的形狀建立全部都是1 的陣列，語法為：

```
np.ones( 陣列形狀 )
```

2.2.2 建立多維陣列

可以使用`np.array()` 函式將多維的串列建立成多維的陣列，屬性`ndim` 顯示陣列的維度，`shape` 顯示陣列的維度形狀，`size` 顯示陣列內所有的元素數量。

例如：用多維串列建立一個2 x 5 (2 row 5 col) 的陣列：

```
[ ] 1 import numpy as np
      2 listdata = [[1,2,3,4,5],
      3               [6,7,8,9,10]]
      4 na = np.array(listdata)
      5 print(na)
      6 print('維度', na.ndim)
      7 print('形狀', na.shape)
      8 print('數量', na.size)
```

```
[[ 1  2  3  4  5]
 [ 6  7  8  9 10]]
維度 2
形狀 (2, 5)
數量 10
```

2.2.3 改變陣列形狀：reshape()

另一種快速建立多維陣列的方式，可以在建立一維陣列後利用`reshape()` 函式改變陣列的形狀。

例如，用`np.arange()` 函式建立一個1 x 16 一維陣列，再利用`reshape()`函式改變成4 x 4 的二維陣列：

```
[ ] 1 import numpy as np
      2 adata = np.arange(1,17)
      3 print(adata)
      4 bdata = adata.reshape(4,4)
      5 print(bdata)
```

```
[ 1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16]
[[ 1  2  3  4]
 [ 5  6  7  8]
 [ 9 10 11 12]
 [13 14 15 16]]
```

2.3 Numpy 陣列取值

2.3.1 一維陣列取值

一維陣列中元素排列的順序就是取值的方式，而這個順序就是索引，語法為：

一維陣列變數 [索引]

也能用起始及終止索引來取得一個範圍的值，語法為：

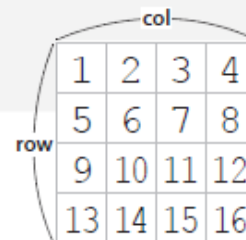
一維陣列變數 [起始索引 : 終止索引 [: 間隔值]]

2.3.2 多維陣列取值

例如，這裡定義一個4 x 4 的陣列：

```
[21] 1 import numpy as np
      2 na = np.arange(1, 17).reshape(4, 4)
      3 na
```

```
array([[ 1,  2,  3,  4],
       [ 5,  6,  7,  8],
       [ 9, 10, 11, 12],
       [13, 14, 15, 16]])
```



1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	16

1. 可以用座標的方式來取得值，如row 索引2，col 索引3 的值：

```
[22] 1 na[2, 3]
```

2. 也可以用row 及col 的索引範圍來取值：

```
[26] 1 print(na[1, 1:3])      #[6,7]
      2 print(na[1:3, 2])    #[7,11]
      3 print(na[1:3, 1:3])  #[[6,7],[7,11]]
      4 print(na[:, 2])      #[[1,3],[9,11]]
      5 print(na[:, 2])      #[3,7,11,15]
      6 print(na[1, :])      #[5,6,7,8]
      7 print(na[:, :])      #矩陣全部
```

2.3.3 產生隨機資料:np.random()

np.random 模組提供了很多方式來生成隨機的資料，以下是常用的函式：

名稱	功能
<code>rand(d0,d1...dn)</code>	依設定維度形狀，返回 0~1 之間的隨機浮點數資料。
<code>randn(d0,d1...dn)</code>	依設定維度形狀，返回標準常態分佈的隨機浮點數資料。
<code>randint(最低[,最高, size])</code>	依設定值範圍（包含最低，不含最高）返回隨機整數資料，size 可設定返回資料的維度形狀。
<code>random(size)</code> <code>random_sample(size)</code> <code>sample(size)</code>	依設定的維度形狀 size 返回隨機資料，返回 0~1 之間的隨機浮點數資料。
<code>choice(陣列, size[, replace])</code>	在指定的陣列中取值，依設定的維度形狀 size 返回隨機資料；陣列若是整數時，結果為 <code>arange(整數)</code> 設定陣列； <code>replace=False</code> 會返回不重複的資料。

注意：size 的設定即為陣列的形狀，其格式可以為串列或是元組。

2.3.4 Numpy 讀取CSV 檔案

在實務中，使用者常將大量的資料儲存在檔案之中，最常見的就是csv 檔。

Numpy可以使用np.genfromtxt() 函式讀取檔案，將內容轉化為陣列。語法如下：

```
np.genfromtxt(檔案名稱, delimiter= 分隔符號, skip_header= 略過列數)
```

2.4 Numpy 的陣列運算功能

2.4.1 Numpy 陣列運算

例如，這裡定義二個3 x 3 的陣列a 與b：

```
[ ] 1 import numpy as np
      2 a = np.arange(1,10).reshape(3,3)
      3 b = np.arange(10,19).reshape(3,3)
      4 print('a 陣列內容：\n', a)
      5 print('b 陣列內容：\n', b)
```

a 陣列內容：

```
[[1 2 3]
 [4 5 6]
 [7 8 9]]
```

b 陣列內容：

```
[[10 11 12]
 [13 14 15]
 [16 17 18]]
```

對單一陣列進行運算

1. 將a 陣列中所有元素都加上一個值：

```
[ ] 1 print('a 陣列元素都加值：\n', a + 1)
```

a 陣列元素都加值：

```
[[ 2  3  4]
 [ 5  6  7]
 [ 8  9 10]]
```

取出指定陣列的元素進行運算

例如將a 陣列中第一個row 或第一個col 加上值：

```
[ ] 1 print('a 陣列取出第一個row都加1: \n', a[0,:] + 1)
    2 print('a 陣列取出第一個col都加1: \n', a[:,0] + 1)
```

a 陣列取出第一個row都加1:

```
[2 3 4]
```

a 陣列取出第一個col都加1:

```
[2 5 8]
```

對多個陣列進行運算

1. 將a、b 陣列中對應元素相加、相乘，要注意陣列的形狀要相同：

```
[ ] 1 print('a b 陣列對應元素相加: \n', a + b)
    2 print('a b 陣列對應元素相乘: \n', a * b)
```

a b 陣列對應元素相加:

```
[[11 13 15]
```

```
[17 19 21]
```

```
[23 25 27]]
```

a b 陣列對應元素相乘:

```
[[ 10  22  36]
```

```
[ 52  70  90]
```

```
[112 136 162]]
```

2. 將a、b 陣列進行內積計算，要使用np.dot() 函式：

```
[ ] 1 print('a b 陣列內積計算: \n', np.dot(a,b))
```

a b 陣列內積計算:

```
[[ 84  90  96]
```

```
[201 216 231]
```

```
[318 342 366]]
```

注意：第一個陣列的 col 與第二個陣列的 row 必須相等才能計算內積，例如第一個陣列為2x3，第二個陣列為3x4。

2.4.2 Numpy 常用的計算及統計函式

下表是常用的Numpy 計算及統計函式：

名稱	說明
sum	加總
prod	乘積
mean	平均值
min	最小值
max	最大值
std	標準差
var	變異數

名稱	說明
median	中位數
argmin	最小元素值索引
argmax	最大元素值索引
cumsum	陣列元素累加
cumprod	陣列元素累積
percentile	以百分比顯示陣列中的指定值
ptp	最大值與最小值的差

2.4.3 Numpy 的排序

Numpy 可以使用`np.sort()` 及`np.argsort()` 函式進行元素的數值及索引的排序。

一維陣列的排序

1. `np.sort()`：可以對陣列中的值進行排序並將結果返回。
2. `np.argsort()`：可以對陣列中的值進行排序並將索引返回。

例如：

```
[ ] 1 import numpy as np
    2 a = np.random.choice(50, size=10, replace=False)
    3 print('排序前的陣列：', a)
    4 print('排序後的陣列：', np.sort(a))
    5 print('排序後的索引：', np.argsort(a))
    6 #用索引到陣列取值
    7 for i in np.argsort(a):
    8     print(a[i], end=',')
```

排序前的陣列： [33 29 9 1 12 10 48 35 15 45]
排序後的陣列： [1 9 10 12 15 29 33 35 45 48]
排序後的索引： [3 2 5 4 8 1 0 7 9 6]
1,9,10,12,15,29,33,35,45,48,

多維陣列的排序

多維陣列的排序方式，可以利用axis 軸向來設定，例如：

```
[ ] 1 import numpy as np
    2 a = np.random.randint(0,10,(3,5))
    3 print('原陣列內容：')
    4 print(a)
    5 print('將每一直行進行排序：')
    6 print(np.sort(a, axis=0))
    7 print('將每一橫列進行排序：')
    8 print(np.sort(a, axis=1))
```

原陣列內容：
[[6 0 4 1 4]
 [8 4 1 8 1]
 [9 3 5 4 1]]
將每一直行進行排序：
[[6 0 1 1 1]
 [8 3 4 4 1]
 [9 4 5 8 4]]
將每一橫列進行排序：
[[0 1 4 4 6]
 [1 1 4 8 8]
 [1 3 4 5 9]]

2.5 Pandas：資料處理分析的強大工具

認識Pandas

Pandas 的特色如下：

1. Pandas 提供兩種主要的資料結構，Series 與 DataFrame。Series 是用來處理序列相關的資料，主要會建立具有索引的一維陣列。DataFrame 則是用來處理結構化的資料，主要是建立具有索引與標籤的二維資料表格。
2. Pandas 簡化了資料的讀取與儲存，無論是由檔案、網頁甚至是資料庫進行資料載入，或是將處理完的資料回存到其他的資源，都提供了方便簡單的方法，讓分析人員更容易處理。
3. 因為整合了Numpy 及Matplotlib 等重要模組，Pandas 在將載入的資料結構化為物件後，即可透過物件所提供的方法進行資料的處理、運算及分析，甚至進一步將資訊進行視覺化的呈現。

安裝Pandas 與載入模組

可以使用下列指令在Python 中安裝Pandas：

```
!pip install pandas
```

使用前請先載入Pandas 模組，一般為了能在使用時更加方便，會設定別名pd：

```
import pandas as pd
```

2.6 Series 的使用

2.6.1 建立Series

使用串列建立Series 物件

Pandas 的Series 是一維的資料陣列，新增的語法為：

```
pd.Series(資料 [,index = 索引])
```

資料可用串列(List)、元組(Tuple)、字典(Dictionary) 或Numpy 的陣列，其中index參數非必填，如果沒有填寫，預設會自動填入由0 開始的整數串列。

使用字典建立Series 物件

如果使用字典來建立Series，字典的鍵(Key) 就會轉換為Series 的索引，而字典的值(Value) 就會成為Series 的資料。

```
[ ] 1 import pandas as pd
     2 dict1 = {'Taipei': '台北', 'Taichung': '台中', 'Kaohsiung':
     3 se = pd.Series(dict1)
     4 print(se)           #顯示Series
     5 print(se.values)    #顯示值
     6 print(se.index)    #顯示索引
     7 print(se['Taipei']) #用索引取值
```

```
Taipei      台北
Taichung    台中
Kaohsiung   高雄
dtype: object
['台北' '台中' '高雄']
Index(['Taipei', 'Taichung', 'Kaohsiung'], dtype='object')
台北
```

2.6.2 Series 資料取值

1. 以索引取值：跟Numpy 一樣，Series 可以使用相關索引來顯示值，例如：

```
[69] 1 import pandas as pd  
      2 se = pd.Series([1,2,3,4,5])  
      3 se[2]
```

2. 自訂索引及取值：設定時可以用index 參數自訂為其他的類型資料。例如：

```
[68] 1 import pandas as pd  
      2 se = pd.Series([1,2,3,4,5], index=['a','b','c','d','e'])  
      3 se['b']
```

2.7 DataFrame 的建立

Pandas 的 DataFrame 是二維的資料陣列，與 Excel 的工作表相同，是使用索引列與欄位組合起來的資料內容。

2.7.1 建立 DataFrame

新增 DataFrame 物件

Pandas 的 DataFrame 新增的語法為：

```
pd.DataFrame( 資料 [ , index = 索引 , columns = 欄位 ] )
```

資料可用串列(List)、元組(Tuple)、字典(Dictionary)、Numpy 陣列，或是組合 Series 物件成為資料來源。index 索引是工作表橫向的列號，columns 是直向的欄位名稱。如果沒有設定 index 或 columns，預設會自動填入由0 開始的整數串列。

設定index 與columns

index 與columns 可以在新增 DataFrame 時依照需求自行設定，例如在剛才的範例中，想要設定學生的姓名為index，而各科的科目為columns：

```
[35] 1 import pandas as pd
      2 df = pd.DataFrame([[65,92,78,83,70],
      3                     [90,72,76,93,56],
      4                     [81,85,91,89,77],
      5                     [79,53,47,94,80]],
      6                     index=['王小明','李小美','陳大同','林小玉'],
      7                     columns=['國文','英文','數學','自然','社會'])
      8 df
```

	國文	英文	數學	自然	社會
王小明	65	92	78	83	70
李小美	90	72	76	93	56
陳大同	81	85	91	89	77
林小玉	79	53	47	94	80

2.7.2 利用字典建立DataFrame

以字典建立DataFrame 也是常用的方式，以剛才的範例來說，改寫成以字典資料格式來新增DataFrame 的方法如下，執行結果與原範例相同：

```
[6] 1 import pandas as pd
      2 scores = {'國文':{'王小明':65,'李小美':90,'陳大同':81,'林小玉':79},
      3          '英文':{'王小明':92,'李小美':72,'陳大同':85,'林小玉':53},
      4          '數學':{'王小明':78,'李小美':76,'陳大同':91,'林小玉':47},
      5          '自然':{'王小明':83,'李小美':93,'陳大同':89,'林小玉':94},
      6          '社會':{'王小明':70,'李小美':56,'陳大同':94,'林小玉':80}}
      7 df = pd.DataFrame(scores)
      8 df
```

	國文	英文	數學	自然	社會
王小明	65	92	78	83	70
李小美	90	72	76	93	56
陳大同	81	85	91	89	94
林小玉	79	53	47	94	80

2.7.3 利用Series 建立DataFrame

利用Series 物件組合成字典

DataFrame 可以由Series 組成的字典資料來新增，例如：

```
[7] 1 import pandas as pd
2 se1 = pd.Series({'王小明':65,'李小美':90,'陳大同':81,'林小玉':79})
3 se2 = pd.Series({'王小明':92,'李小美':72,'陳大同':85,'林小玉':53})
4 se3 = pd.Series({'王小明':78,'李小美':76,'陳大同':91,'林小玉':47})
5 se4 = pd.Series({'王小明':83,'李小美':93,'陳大同':89,'林小玉':94})
6 se5 = pd.Series({'王小明':70,'李小美':56,'陳大同':94,'林小玉':80})
7 df = pd.DataFrame({'國文':se1,'英文':se2,'數學':se3,
8                    '自然':se4,'社會':se5})
9 df
```

	國文	英文	數學	自然	社會
王小明	65	92	78	83	70
李小美	90	72	76	93	56
陳大同	81	85	91	89	94
林小玉	79	53	47	94	80

使用pd.concat() 函式合併Series 物件

可以使用pd.concat() 函式將多個Series 合併成DataFrame，例如：

```
[13] 1 import pandas as pd
2 se1 = pd.Series({'王小明':65,'李小美':90,'陳大同':81,'林小玉':79})
3 se2 = pd.Series({'王小明':92,'李小美':72,'陳大同':85,'林小玉':53})
4 se3 = pd.Series({'王小明':78,'李小美':76,'陳大同':91,'林小玉':47})
5 se4 = pd.Series({'王小明':83,'李小美':93,'陳大同':89,'林小玉':94})
6 se5 = pd.Series({'王小明':70,'李小美':56,'陳大同':94,'林小玉':80})
7 df = pd.concat([se1,se2,se3,se4,se5], axis=1)
8 df.columns=['國文','英文','數學','自然','社會']
9 df
```

	國文	英文	數學	自然	社會
王小明	65	92	78	83	70
李小美	90	72	76	93	56
陳大同	81	85	91	89	94
林小玉	79	53	47	94	80

2.8 Pandas DataFrame 資料取值

2.8.1 DataFrame 基本取值

指定欄位名稱取值

可以利用DataFrame 的欄位名稱取得該欄的值，語法如下：

```
df[ 欄位名稱 ]
```

例如，取得所有學生自然科成績，回傳值是一個Series：

```
[14] 1  df["自然"]  
  
王小明    83  
李小美    93  
陳大同    89  
林小玉    94  
Name: 自然, dtype: int64
```

指定多個欄位名稱取值

若要取得 2 個以上欄位資料，則需將欄位名稱化為串列，語法為：

```
df[[ 欄位名稱 1, 欄位名稱 2, ...]]
```

例如，取得所有學生的國文、數學及自然科成績：

```
[15] 1 df[["國文", "數學", "自然"]]
```

	國文	數學	自然
王小明	65	78	83
李小美	90	76	93
陳大同	81	91	89
林小玉	79	47	94

指定欄位以條件式進行判斷取值

也可以將欄位進行條件式判斷後根據回傳的布林值(為True 時)來取得資料，語法為：

```
df[ 欄位判斷式 ]
```

例如，取得國文科成績80 分以上 (含) 的所有學生成績：

```
[16] 1 df[df["國文"] >= 80]
```

	國文	英文	數學	自然	社會
李小美	90	72	76	93	56
陳大同	81	85	91	89	94

以values 屬性取得資料

DataFrame 的values 屬性可取得全部資料，返回是一個二維串列，語法為：

```
df.values
```

1. 可以直接用values 屬性以二維串列的格式取得所有的值，以剛才的範例來看：

```
[17] 1 df.values  
array([[65, 92, 78, 83, 70],  
       [90, 72, 76, 93, 56],  
       [81, 85, 91, 89, 94],  
       [79, 53, 47, 94, 80]])
```

2.8.2 以索引及欄位名稱取得資料：df.loc()

df.loc() 函式可直接以索引及欄位名稱取得資料，很容易理解，使用上也較為方便，語法為：

```
df.loc[索引名稱, 欄位名稱]
```

1. 例如，要取得 林小玉 的 社會 科目成績：

```
[36] 1 df.loc["林小玉", "社會"]
```

2.8.3 以索引及欄位編號取得資料：df.iloc()

iloc 可直接以索引及欄位的索引編號(由0開始)取得資料，使用的語法為：

```
df.iloc[ 索引編號 , 欄位編號 ]
```

iloc 的使用方式與loc大致相同，只要將索引及欄位的「名稱」改為「編號」即可。

1. 例如，要取得 **林小玉** 的 **社會** 科目成績：

```
[44] 1 df.iloc[3, 4]
```

80

2.8.4 取得最前或最後幾列資料

取得最前幾列的資料：df.head()

如果要取得最前面幾列資料，可使用 df.head() 方法，語法為：

```
df.head([n])
```

參數 n 非必填，表示取得最前面 n 列資料，若省略預設會取得 5 筆資料。

取得後幾列的資料：df.tail()

若要取得最後面幾列資料，則使用 df.tail() 方法，語法為：

```
df.tail([n])
```

使用方法與 head 相同。

2.9 DataFrame 資料操作

2.9.1 DataFrame 資料排序

依值排序：df.sort_values()

首先根據DataFrame 資料數值排序，語法為：

```
df.sort_values(by = 欄位[, ascending = 布林值])
```

- **by**：做為排序值的欄位名稱。
- **ascending**：可省略，True 表示遞增排序 (預設值)，False 表示遞減排序。

依索引排序：df.sort_index()

接著是根據軸向(橫列或直欄)排序，語法為：

```
df.sort_index(axis=軸向編號[, ascending=布林值])
```

- **axis**：軸向編號，0 表示依索引名稱 (橫列) 排序，1 表示依欄位名稱 (直欄) 排序。

2.9.2 DataFrame 資料修改

要修改 DataFrame 的資料非常簡單，只要先指定資料項目所在位置，再設定指定值即可。

例如，修改 **王小明** 的 **數學** 成績為 90：

```
[56] 1 df.loc["王小明"]["數學"] = 90  
      2 df
```

	國文	英文	數學	自然	社會
王小明	65	92	90	83	70
李小美	90	72	76	93	56
陳大同	81	85	91	89	77
林小玉	79	53	47	94	80

2.9.3 刪除 DataFrame 資料

DataFrame 可以使用 drop 方法來刪除資料，語法為：

```
資料變數 = df.drop(索引或欄位名稱 [, axis=軸向編號])
```

- **axis**：軸向編號，0 表示依索引名稱（橫列）刪除，1 表示依欄位名稱（直欄）刪除。若沒有填寫，預設值為 0。

刪除橫列的資料

例如，刪除 王小明 的成績：

```
[58] 1 df.drop("王小明")
```

	國文	英文	數學	自然	社會
李小美	90	72	76	93	56
陳大同	81	85	91	89	77
林小玉	79	53	47	94	80

刪除直欄的資料

例如，刪除所有人的 數學 成績：

```
[59] 1 df.drop("數學", axis=1)
```

	國文	英文	自然	社會
王小明	80	80	80	80
李小美	90	72	93	56
陳大同	81	85	89	77
林小玉	79	53	94	80

刪除連續橫列範圍的資料

若要刪除連續多列的資料，可使用刪除「範圍」方式處理，語法為：

```
df.drop(df.index[ 啟始編號:終止編號 ], axis= 軸向編號 )
```

指定二個索引編號之間的範圍資料，是用「啟始編號：終止編號」來表示，因為是不包含終止編號所代表的項目，所以設定時終止編號要加1。

刪除連續直欄範圍的資料

刪除連續直欄的語法為：

```
df.drop(df.columns[ 啟始編號:終止編號 ], axis= 軸向編號 )
```